

Ecommerce API

Training Documentation

A complete 4-week hands-on backend training program

Node.js • Express.js • MongoDB • Mongoose

Duration:

4 Weeks

Level: Intermediate to Advanced

1. Project Overview

This is a full-featured RESTful Ecommerce API built with Node.js, Express, and MongoDB. It covers the complete lifecycle of an online store — from user registration and product management to cart, orders, payments, and an admin dashboard.

The project is designed as a 4-week hands-on training. By the end of it, students will have built a production-ready backend from scratch and gained practical experience with all the tools and patterns used in real-world Node.js projects.

Learning Objectives

- Build RESTful APIs from scratch using Node.js and Express
- Design and work with MongoDB schemas using Mongoose ODM
- Implement Authentication and Authorization with JWT and bcrypt
- Upload and manage images with Cloudinary
- Build a complete shopping cart system with coupons and stock management
- Integrate Stripe for online payments including webhooks
- Use Mongoose Transactions for atomic, multi-step database operations
- Send automated emails (order confirmation, OTP, status updates) via Nodemailer
- Build an Admin Dashboard with real statistics using MongoDB Aggregation Pipeline
- Apply proper error handling, input validation, and role-based access control

2. Tech Stack

Technology	Purpose
Node.js	Server-side JavaScript runtime environment
Express.js	Web framework — routing, middleware, error handling
MongoDB	NoSQL document database
Mongoose	ODM — schemas, models, validation, hooks, methods
JWT	Stateless authentication tokens
bcryptjs	Secure password hashing
Cloudinary	Cloud-based image storage and delivery
Multer	Middleware for handling multipart/form-data (file uploads)

Technology	Purpose
Nodemailer	Email sending (OTP, order confirmations, status updates)
Slugify	Auto-generate URL-friendly slugs from product names
dotenv	Load environment variables from .env file
cors	Cross-Origin Resource Sharing middleware
cookie-parser	Parse cookies for JWT refresh token handling
Joi	For make validation
Morgan	For manage the all requests

3. Project Architecture

The project follows the MVC (Model-View-Controller) pattern with a clear separation of concerns across layers:

```
ecommerce-api/  
├─ config/           → Cloudinary setup  
├─ models/           → Mongoose schemas and models  
│   ├── User.model.js  
│   ├── Product.model.js  
│   ├── Order.model.js  
│   ├── Cart.model.js  
│   ├── Wishlist.model.js  
│   └── OTP.model.js  
├─ controllers/      → Business logic for every resource  
├─ DB/               → Database connection  
├─ routes/           → Express route definitions  
├─ middleware/       → Auth guard, role check, upload, error handler , validate  
├─ utils/            → sendEmail, uploadToCloudinary,  
├─ validation/       → using joi create the files  
└─ index.js          → App entry point  
└─ .env              → all environment variables  
└─ vercel.json       → handle the api deploy to vercel
```

4. Database Models

4.1 User Model

Represents a system user — either a customer or an admin. Supports email verification via OTP, password reset, multiple shipping addresses, and a wishlist.

Field	Type	Description	Required
username	String	Display name — required	Yes
email	String	Unique, lowercase — used as the login identifier	Yes
password	String	Stored as a bcrypt hash. select: false hides it from queries	Yes
phone	String	Optional phone number	No
avatar	String	Profile picture URL (defaults to a placeholder icon)	No
role	String	"admin" or "customer" — controls access (default: customer)	No
addresses	[Address]	Array of shipping address sub-documents	No
wishlist	[ObjectId]	References to Product — the user's saved items	No
isVerified	Boolean	Whether the email has been verified via OTP (default: false)	No
resetPasswordToken	String	Token issued for password recovery flow	No
resetPasswordExpire	Date	Expiry timestamp for the reset token	No

Hooks & Methods

- `pre('save')` — Automatically hashes the password with bcrypt before saving to the database
- `comparePassword(enteredPassword)` — Instance method that securely compares a plain-text password against the stored hash

4.2 Product Model

Represents a store product with full details — pricing, inventory, images, reviews, and categorization.

Field	Type	Description	Required
name	String	Product name (maxlength: 200)	Yes
slug	String	URL-friendly version of the name — auto-generated via Slugify	Auto
shortDescription	String	Brief summary shown in listings (maxlength: 500)	Yes
description	String	Full detailed description	Yes
price	Number	Original price (min: 0)	Yes
discountPrice	Number	Sale price — 0 means no discount is active	No
stock	Number	Units available in inventory	Yes
sku	String	Unique Stock Keeping Unit code	No
images	[Image]	Array of {public_id, url} objects from Cloudinary (min: 1)	Yes
category	String	Primary category (lowercase)	Yes
subcategory	String	Optional secondary category	No
brand	String	Brand name	No
tags	[String]	Keywords used for search and filtering	No
reviews	[Review]	Embedded review sub-documents (user, rating, comment)	No
averageRating	Number	Auto-calculated average from all reviews	Auto
numReviews	Number	Auto-calculated count of reviews	Auto
featured	Boolean	Marks the product as featured (default: false)	No
isActive	Boolean	Whether the product is visible to customers (default: true)	No
createdBy	ObjectId	Reference to the admin User who created the product	Yes

Hooks, Methods & Indexes

- `pre('save')` — Generates a unique slug from the product name whenever the name is modified
- `calcAverageRating()` — Recalculates `averageRating` and `numReviews` after a review is added or deleted

- Text Index on (name, description, brand) — enables full-text search queries
- Additional indexes on category, brand, price, averageRating, createdAt for fast filtering and sorting

4.3 Order Model

Represents a customer purchase. Tracks all financial details, shipping, payment, and the full status lifecycle from placement to delivery.

Field	Type	Description	Required
user	ObjectId	Reference to the customer who placed the order	Yes
items	[OrderItem]	Array of purchased items (name, image, price, quantity snapshot at time of purchase)	Yes
shippingAddress	Object	fullName, phone, country, city, address, postalCode	Yes
paymentMethod	String	"cash", "stripe", "paypal", or "paymob" (default: cash)	No
paymentStatus	String	"pending", "paid", "failed", or "refunded"	Auto
transactionId	String	Stripe PaymentIntent ID — used to match webhook events	Auto
subtotal	Number	Sum of all item prices before fees and discounts	Yes
shippingFee	Number	0 if subtotal >= 1000 EGP, otherwise 50 EGP	Auto
tax	Number	14% VAT applied to the subtotal	Auto
discount	Number	Coupon discount amount applied to the order	Auto
totalPrice	Number	Final amount: subtotal + shippingFee + tax - discount	Yes
status	String	pending > confirmed > processing > shipped > delivered / cancelled / returned	Auto
paidAt	Date	Timestamp set when payment is confirmed	Auto
deliveredAt	Date	Timestamp set when order is marked delivered	Auto
cancelledAt	Date	Timestamp set when order is cancelled	Auto

Field	Type	Description	Required
customerNote	String	Optional note from the customer (maxlength: 1000)	No
adminNote	String	Internal note from the admin (maxlength: 1000)	No

4.4 Cart Model

Represents a user's shopping cart. One cart per user (unique). Uses Mongoose Virtuals to compute totals dynamically without storing them in the database.

Field	Type	Description	Required
user	ObjectId	Unique reference to the cart's owner	Yes
items	[CartItem]	Array of {product, name, image, price, quantity}	No
coupon.code	String	Applied coupon code (uppercase)	No
coupon.discountType	String	"percentage" or "fixed"	No
coupon.discountValue	Number	The discount percentage or fixed amount	No
subtotal (virtual)	Number	Computed: sum of (price x quantity) for all items	Virtual
discountAmount (virtual)	Number	Computed: coupon saving based on type and value	Virtual
total (virtual)	Number	Computed: subtotal minus discountAmount	Virtual
itemCount (virtual)	Number	Computed: total number of units across all items	Virtual

Available Coupons

Coupons are defined as a static object in the Cart controller:

Code	Type	Value
SAVE10	Percentage	10% off the subtotal
SAVE20	Percentage	20% off the subtotal
SAVE50	Percentage	50% off the subtotal
SAVE80	Percentage	80% off the subtotal

Code	Type	Value
OFF50	Fixed	50 EGP off the subtotal

4.5 Wishlist Model

Stores a user's saved products. One wishlist per user. A pre-find hook auto-populates product details on every query.

Field	Type	Description	Required
user	ObjectId	Unique reference to the wishlist owner (index)	Yes
products	[ObjectId]	Array of Product references	No

4.6 OTP Model

Temporarily stores the OTP code and user registration data until the email is verified. Once verified, the user account is created and this document is deleted.

Field	Type	Description	Required
email	String	The email address being verified	Yes
otp	String	The one-time passcode (typically hashed before storing)	Yes
expiresAt	Date	Expiry time for the OTP — expired codes are rejected	Yes
userData	Object	Temporary storage for the user's registration data pending verification	No

5. API Endpoints

Base URL: <https://your-api.com/> | Authentication Header: Authorization: Bearer <token>

5.1 Authentication /auth

Method	Endpoint	Description	Auth
POST	/auth/register/send-otp	Register a new user — sends a verification OTP by email	Public
POST	/auth/verify-otp	Verify the OTP and activate the account	Public
POST	/auth/login	Log in — returns a signed JWT	Public
POST	/auth/logout	Log out the current user	User
POST	/auth/forgot-password/send-otp	Request a password reset link or token	Public
POST	/auth/forgot-password/verify-otp	Set a new password using the reset token	Public
GET	/auth/me	Get the authenticated user's profile	User

5.1 Users /users

Method	Endpoint	Description	Auth
POST	/users/add	add a new user from admin panel	Admin
GET	/users/all	Get all users – admin panel	Admin
GET	/users/:id	Return one user	Admin
PATCH	/users/:id	Update the user data	User
DELETE	/users/:id	Delete one user – admin panel	Admin

5.2 Products /products

Method	Endpoint	Description	Auth
GET	/products	Get all active products — supports pagination, category, brand, price filters, and sorting	Public
GET	/products/search	Advanced search — text, category, subcategory, brand, tags, price range, sort, pagination	Public
GET	/products/:id	Get a single product by ID	Public
POST	/products	Create a new product — accepts multipart/form-data with image uploads to Cloudinary	Admin
PUT	/products/update/:id	Update a product — can delete specific images and upload new ones	Admin
DELETE	/products/:id	Delete a product and remove all its images from Cloudinary	Admin
POST	/products/:id/reviews	Add a review to a product (one review per user per product)	User
DELETE	/products/:id/reviews/:rid	Delete a review — allowed only for the review owner or an admin	User/Admin
GET	/products/:id/reviews	Get all reviews for a product	Public

5.3 Cart /carts

Method	Endpoint	Description	Auth
GET	/carts	Get the cart — creates one automatically if it does not exist	User
POST	/carts/items	Add an item to the cart — deducts stock immediately	User
PATCH	/carts/items	Update the quantity of an item — adjusts stock by the difference	User
DELETE	/carts/items/:productId	Remove an item from the cart — restores its stock	User
POST	/carts/coupon	Apply a discount coupon to the cart	User
DELETE	/carts/coupon	Remove the currently applied coupon	User
DELETE	/carts/clear	Clear all items and the coupon from the cart	User

5.4 Orders `/orders`

Method	Endpoint	Description	Auth
POST	<code>/orders</code>	Create a new order (cash or Stripe) — uses a Mongoose Transaction	User
GET	<code>/orders/my</code>	Get the authenticated user's orders — supports pagination and status filter	User
GET	<code>/orders/my/:id</code>	Get details of a specific order (owner only)	User
PATCH	<code>/my/:id/cancel</code>	Cancel an order — only allowed if status is pending or confirmed. Restores stock	User
GET	<code>/admin/dashboard</code>	Admin: get all orders — supports filters by status, payment, date range, and sorting	Admin
GET	<code>/admin/carts</code>	Admin: get full details of any order	Admin
GET	<code>/admin</code>	Admin: all orders (filterable)	Admin
GET	<code>/admin/id</code>	Admin: get admin order	Admin
PATCH	<code>/admin/:id/status</code>	Admin: update order status and send an automated email to the customer	Admin

5.5 Wishlist `/wishlists`

Method	Endpoint	Description	Auth
GET	<code>/wishlists/my</code>	Get the user's wishlist with full product details	User
POST	<code>/wishlists/add/:productId</code>	Add a product to the wishlist	User
DELETE	<code>/wishlists/remove/:productId</code>	Remove a product from the wishlist	User
DELETE	<code>/wishlists/clear</code>	Clear the entire wishlist	User

5.6 Admin — Dashboard & Analytics

Method	Endpoint	Description	Auth
GET	/admin/dashboard	Full stats: revenue, order counts, top products, daily revenue (last 7 days), recent orders, total customers	Admin
GET	/admin/carts	View all active carts with user info and item details	Admin
GET	/admin/wishlists	View all user wishlists with pagination	Admin
GET	/admin/wishlists/stats	Get the top 10 most wishlisted products	Admin

6. Key Technical Features

6.1 Mongoose Transactions

createOrder and cancelOrder wrap all database writes in a single atomic transaction. If any step fails, every change is rolled back:

- Validates stock for each product before creating the order
- Creates the order document and deducts stock in the same operation
- Clears the cart only after the order is successfully persisted
- Restores stock for every item when an order is cancelled

6.2 Cloudinary Image Management

Product images are stored in Cloudinary. Each image is saved as an object containing a public_id and a url:

- Multiple images can be uploaded when creating a product
- On update: specific images can be deleted by public_id and new ones uploaded simultaneously
- On product delete: all images are removed from Cloudinary before the document is deleted
- Multer handles the multipart form data and streams files to the upload utility

6.3 Smart Cart & Stock Management

The cart automatically manages inventory in real time:

- Adding an item to the cart immediately reduces the product's stock
- Updating the quantity adjusts stock by the exact difference (increase or decrease)
- Removing an item from the cart returns its quantity to stock
- subtotal, total, discountAmount, and itemCount are Mongoose Virtuals — computed on the fly, never stored

6.4 Automated Email System

The API sends transactional emails automatically in the following scenarios:

- Order placement confirmation — includes an itemized table and full price breakdown
- Order status updates — triggered for every status change (confirmed, shipped, delivered, cancelled, returned)
- Email verification OTP — sent during the registration flow

6.6 Admin Dashboard Analytics

The dashboard endpoint runs multiple MongoDB Aggregation Pipelines in parallel to return:

- Total, monthly, and last-month revenue — with a month-over-month growth percentage
- Order counts broken down by every status (pending, processing, shipped, delivered, cancelled, etc.)
- Top 5 best-selling products by units sold and revenue generated
- Daily revenue and order count for the past 7 days
- The 5 most recent orders
- Total registered customer count

7. 4-Week Training Plan

The project is split into four focused weeks. Each week builds on the previous one and ends with concrete deliverables that students submit for review.

Week	Topics & Deliverables
Week 1	Node.js, Express, MongoDB, Mongoose Basics — Project setup, environment configuration, building all six database schemas (User, Product, Order, Cart, Wishlist, OTP), understanding Schema hooks and methods, MongoDB connection and error handling.
Week 2	Authentication System — JWT generation and verification, bcrypt password hashing, OTP email verification flow, Auth and Admin middleware, Forgot Password / Reset Password flow, Profile endpoints.

Week	Topics & Deliverables
Week 3	Products, Cart & Wishlist — Cloudinary image upload, full Product CRUD with image management, advanced search and filtering, complete Cart system with stock management, coupon system, Mongoose Virtuals, Wishlist endpoints.
Week 4	Orders, Payments & Admin Dashboard — createOrder with Mongoose Transactions, Stripe PaymentIntent and Webhook, Nodemailer automated emails, Admin order management, Admin Dashboard with MongoDB Aggregation Pipeline statistics.

Weekly Deliverables

Week 1

1. Initialize the project (npm init, .env setup, folder structure)
2. Connect to MongoDB Atlas and handle connection errors
3. Build the User Schema with the pre-save password hashing hook
4. Build the Product Schema with the pre-save slug hook and calcAverageRating method
5. Build the Cart, Order, Wishlist, and OTP schemas

Week 2

6. Implement Register with OTP generation and email sending
7. Implement Verify OTP to activate the user account
8. Implement Login with JWT generation
9. Write the protect middleware (verify JWT) and adminOnly middleware
10. Implement the full Forgot Password and Reset Password flow
11. Build the Profile GET, UPDATE, and change-password endpoints

Week 3

12. Implement all Product endpoints with Cloudinary upload via Multer
13. Implement the Search endpoint with all filter and sort options
14. Build the complete Cart system with real-time stock management
15. Add the coupon system with apply and remove endpoints
16. Build all Wishlist endpoints

Week 4

17. Implement createOrder using a Mongoose Transaction
18. Add Stripe PaymentIntent creation for online payments
19. Integrate Nodemailer for all transactional emails
20. Build the Admin Dashboard with Aggregation Pipeline queries
21. Build Admin CRUD for Users and Admin Order management endpoints

8. Environment Variables

Important: never commit your .env file to Git. Add it to .gitignore immediately after creation.

```
# Server
PORT=5000
NODE_ENV=development
```

```
# Database
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/ecommerce
```

```
# JWT
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRE=7d
```

```
# Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

```
# Email (Nodemailer)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password
```

9. Evaluation Criteria

Criterion	Score	Details
All Endpoints Working	25%	Every API route returns correct responses and status codes
Business Logic Correctness	20%	Stock management, order flow, coupon calculations all behave correctly
Authentication & Authorization	15%	JWT, OTP flow, role-based middleware all function properly
Error Handling	15%	Meaningful error messages with appropriate HTTP status codes

Criterion	Score	Details
Cloudinary Integration	10%	Image upload, partial update (add/delete), and full delete on product removal
Code Quality & Structure	5%	Clean, consistent, well-organized code following the project's architecture

Ecommerce API Training Documentation | Confidential — Internal Use Only